

One release.
Four spec types.
One BCC gate.

How a large enterprise catches breaking contract changes before they reach consumers.

OPENAPI

GRAPHQL

gRPC

ASYNCAPI

[same question →](#)

Every change is reasonable. The risk is compatibility.

A global order-fulfillment release is evolving several consumer-facing contracts at once.

OpenAPI	—————	Order history is growing, so clients need pagination.
GraphQL	—————	The customer portal is standardizing terminology.
gRPC	—————	International warehouses need globally unique order IDs.
AsyncAPI	—————	Shipping partners need richer shipment status information.

The business need is valid. Existing consumers still depend on the current contract.

Different spec types create the same release responsibility.

SPEC TYPE	CONSUMER SURFACE	CHANGE TO TEST
OpenAPI	Order-history clients	Make limit mandatory
GraphQL	Portal queries	Rename customerName
gRPC	Warehouse clients	int64 → string ID
AsyncAPI	Event consumers	string → status object

Will existing consumers continue to work?

The first demo makes the compatibility rule visible.

Pagination gives us one safe change and one breaking change that look almost identical in the spec.

SAFE

Add an optional query parameter

```
- name: skip  
  in: query  
  required: false  
  default: 0
```

COMPATIBLE

BREAKING

Make a new query parameter mandatory

```
- name: limit  
  in: query  
  required: true  
  default: 20
```

INCOMPATIBLE

The demo shows the exact change, command, and verdict.

01	————	Trusted baseline	Committed spec in specs/baseline/
02	————	Business-driven edit	openapi_change.sh --add-skip
03	————	Docker BCC check	run-bcc.sh openapi --add-skip
04	————	Visible result	COMPATIBLE or INCOMPATIBLE

The audience never has to learn four tools. They see one compatibility workflow applied across four spec types.

The same gate catches different consumer breakage.

GraphQL

Portal query selects
customerName



Rename it to buyerName

Existing field selection no
longer matches

gRPC

Warehouse client expects int64
order_id



Change it to string

Generated clients expect a
number

AsyncAPI

Partner reads status as a string



Make status an object

Existing event payload shape
changes

Different syntax. Different consumers. One question about backward compatibility.

Local hooks give fast feedback. CI protects the shared branch.

LOCAL DEMONSTRATION

```
git commit --no-verify  
# intentional demo commit
```

Use this only when you intentionally want the breaking commit to reach CI.

PULL REQUEST GATE

```
backward-compatibility-check  
--base-branch origin/main  
--target-path ../orders.yaml
```

INCOMPATIBLE → CI FAILS

The commit is allowed into the branch for the demo; the shared gate rejects the release path.

A short path from business change to release decision.

```
cd bcc-enterprise-demo

./scripts/run-bcc.sh openapi --add-skip
./scripts/run-bcc.sh openapi --add-limit

./scripts/cleanup.sh
```

Then show the same pattern for:

GraphQL
field rename

gRPC
ID type change

AsyncAPI
payload shape change

One enterprise release. Several legitimate changes. One compatibility gate.